КАК ЗАПУСКАТЬ ПРОЕКТ

Пошаговая инструкция для Windows-ПК с Visual Studio: запуск готового релиза, сборка из исходников, создание обычных отчетов, HTML-отчетов и импорт данных в Ghidra.

Дата подготовки: 02.07.2026

0. Что это за проект и какие отчеты он делает

EH/Unwind Analyzer анализирует Windows PE32+ AMD64 файлы: EXE, DLL, PYD и другие PE-файлы. Главная область проекта - exception/unwind metadata: .pdata, RUNTIME_FUNCTION и UNWIND_INFO. Статический режим просто читает файл как байты. Динамический режим запускает целевой EXE под Windows Debug API.

Консольный отчет	Печатается прямо в терминал. Его удобно сохранить в TXT через перенаправление > file.txt 2>&1.
JSON	Полный машинный отчет: метаданные, функции, эвристики, динамика, итоговая оценка. Открывается в VS Code, Notepad++, браузере или валидаторах.
HTML	Самодостаточный интерактивный отчет. Открывается двойным кликом или командой start. Интернет не нужен.
CSV	Таблица функций и unwind-данных. Удобно открывать в Excel или LibreOffice Calc.
Ghidra CSV	Специальный CSV для скрипта ghidra/import_unwind_csv.py. Нужен для разметки функций, комментариев и warning bookmarks в Ghidra.

Важно по безопасности: статический анализ не исполняет файл. Динамический режим с --dynamic исполняет целевой EXE. Для чужих или подозрительных файлов используйте отдельную Windows VM.

Что считается самым быстрым путем

Если нужно просто пользоваться проектом и смотреть отчеты, быстрее всего взять готовый архив release-output\eh-unwind-analyzer-1.0.0-windows-x64.zip, распаковать его и запускать bin\eh_unwind_analyzer.exe. Сборка через Visual Studio нужна, если преподаватель или заказчик хочет увидеть, что проект собирается из исходников.

1. Подготовка Windows-ПК

1.1. Что должно быть установлено

- Windows 10/11 x64.

- Visual Studio 2022 с workload Desktop development with C++.

- Компонент MSVC v143 x64/x86 build tools.

- Windows SDK, который ставится вместе с C++ workload.

- CMake. Он часто ставится вместе с Visual Studio, но можно поставить отдельно.

- Python 3, если нужно запускать тестовые Python-скрипты и валидаторы.

- Ghidra, если нужно импортировать Ghidra CSV и смотреть функции в CodeBrowser.

- Браузер: Edge, Chrome или Firefox для HTML-отчетов.

1.2. Как открыть правильный терминал Visual Studio

Для сборки MSVC удобнее всего использовать не обычный cmd, а x64 Native Tools Command Prompt for VS 2022. Он заранее настраивает PATH, INCLUDE, LIB и другие переменные так, чтобы cl.exe был доступен.

```
Пуск -> Visual Studio 2022 -> x64 Native Tools Command Prompt for VS 2022
```

В открывшемся окне проверьте инструменты:

```
cl
cmake --version
python --version
```

Нормально, если команда cl без аргументов выводит сведения о компиляторе и ошибку про отсутствие входных файлов. Плохо, если написано, что cl не найден. Тогда открыт не тот терминал или в Visual Studio не установлен C++ workload.

1.3. Как перейти в папку проекта

В командах ниже используется переменная PROJECT. Замените путь на тот, где реально лежит папка ip_full. Если путь содержит пробелы или русские буквы, обязательно оставляйте кавычки.

```
set PROJECT=C:\путь\к\ip_full
cd /d "%PROJECT%"
dir
```

Для текущей папки на этом компьютере пример выглядит так:

```
cd /d "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"
```

> Совет: для динамического режима иногда проще сделать короткий ASCII-путь, например C:\EHUA\ip_full. Статический анализ обычно нормально работает и с русскими путями, но для отладки Windows меньше сюрпризов, когда путь короткий и без спецсимволов.

2. Способ A: запуск готового Windows-релиза без сборки

Этот способ нужен, если проект уже передан как готовый комплект и хочется сразу анализировать файлы. Компилятор здесь не нужен. Нужен только распакованный релизный архив.

2.1. Распаковать архив

```
cd /d "%PROJECT%"
cd release-output
powershell -NoProfile -Command "Expand-Archive -Force 'eh-unwind-analyzer-1.0.0-windows-x64.zip'
    'eh-unwind-analyzer-1.0.0-windows-x64'"
cd eh-unwind-analyzer-1.0.0-windows-x64
dir
```

После распаковки исполняемый файл находится здесь:

```
bin\eh_unwind_analyzer.exe
```

2.2. Проверить версию и справку

```
set TOOL=%CD%\bin\eh_unwind_analyzer.exe
"%TOOL%" --version
"%TOOL%" --help
```

В версии должно быть product 1.0.0 и report schema 1.1.0. В справке должны быть опции --json, --html, --csv, --ghidra-csv, --dynamic, --timeout-ms и --trace-steps.

2.3. Сделать отдельную папку для своих отчетов

```
mkdir my_reports
```

Лучше не сохранять новые отчеты прямо в bin или docs. Отдельная папка my_reports упрощает показ результата: в одном месте лежат TXT, JSON, HTML и CSV.

2.4. Запустить простой статический анализ

```
set TARGET=C:\путь\к\target.exe
"%TOOL%" "%TARGET%" --heuristics
```

Этот запуск ничего не записывает на диск. Он выводит результат в терминал: количество RUNTIME_FUNCTION, ошибки разбора, findings, score и verdict. Это удобно для быстрой проверки.

2.5. Сохранить обычный консольный отчет в TXT

```
"%TOOL%" "%TARGET%" --heuristics > "my_reports\console_report.txt" 2>&1
notepad "my_reports\console_report.txt"
```

Конструкция > file.txt 2>&1 сохраняет и обычный вывод, и ошибки в один файл. Такой TXT удобно отправлять вместе с HTML и JSON.

2.6. Сгенерировать полный набор отчетов: JSON, HTML, CSV

```
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\report.json" ^
  --html "my_reports\report.html" ^
  --csv "my_reports\functions.csv"
```

Открытие результатов:

```
notepad "my_reports\report.json"
start "" "my_reports\report.html"
start "" "my_reports\functions.csv"
```

HTML-отчет самодостаточный: внутри уже есть стили и скрипты, интернет не нужен. CSV обычно открывается в Excel. JSON лучше смотреть в VS Code, Notepad++ или любом редакторе с подсветкой JSON.

2.7. Получить подробный dump RUNTIME_FUNCTION и unwind operations

```
"%TOOL%" "%TARGET%" --dump --limit 200 --ops > "my_reports\runtime_functions.txt" 2>&1
notepad "my_reports\runtime_functions.txt"
```

Опция --dump печатает записи RUNTIME_FUNCTION. --limit 200 ограничивает вывод первыми 200 записями, а --ops добавляет декодированные unwind operations. Это полезно, если нужно показать не только итоговый score, но и низкоуровневые данные.

3. Способ B: сборка из исходников через Visual Studio

3.1. Самый простой MSVC-вариант: build_msvc.bat

Откройте x64 Native Tools Command Prompt for VS 2022, перейдите в корень ip_full и запустите готовый bat-файл. Он вызывает cl.exe напрямую и кладет EXE в build\msvc-local.

```
cd /d "%PROJECT%"
build_msvc.bat
set TOOL=%CD%\build\msvc-local\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Если появляется сообщение cl.exe not found, значит терминал открыт неправильно. Нужно именно x64 Native Tools Command Prompt, а не обычный cmd.

3.2. Полная сборка Visual Studio через CMake

Этот вариант лучше показывать как официальную сборку проекта: CMake создает Visual Studio build tree, затем собирается Release, после чего запускаются CTest-проверки.

```
cd /d "%PROJECT%"
cmake -S . -B build\msvc -G "Visual Studio 17 2022" -A x64 -DBUILD_TESTING=ON
cmake --build build\msvc --config Release --parallel
ctest --test-dir build\msvc -C Release --output-on-failure
set TOOL=%CD%\build\msvc\Release\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Нормальный результат ctest - все тесты проходят. В релизном паспорте проекта зафиксировано 16/16 PASS для MSVC и 16/16 PASS для MinGW.

3.3. Сборка через CMake presets

В проекте есть CMakePresets.json. Пресеты используют генератор Ninja. Этот путь удобен, если Ninja установлен и доступен в PATH.

```
cd /d "%PROJECT%"
cmake --preset release
cmake --build --preset release
ctest --preset release
set TOOL=%CD%\build\release\eh_unwind_analyzer.exe
"%TOOL%" --version
```

3.4. MinGW-вариант, если нужен именно MinGW

```
cd /d "%PROJECT%"
build_mingw.bat
set TOOL=%CD%\build\mingw-local\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Для build_mingw.bat нужен gcc из MinGW-w64 в PATH. Если gcc лежит не в PATH, можно задать переменную GCC полным путем к gcc.exe.

4. Практический маршрут: получить отчеты на встроенных примерах

В исходной папке есть тестовые PE-файлы. На них удобно проверять работу без поиска сторонних EXE. Ниже команды используют переменную TOOL, которую вы задали после сборки или после распаковки релиза.

4.1. Проверить benign_minimal.exe

```
cd /d "%PROJECT%"
mkdir my_reports
set TARGET=%CD%\tests\samples\bin\benign_minimal.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\benign_minimal.json" ^
  --html "my_reports\benign_minimal.html" ^
  --csv "my_reports\benign_minimal_functions.csv"
start "" "my_reports\benign_minimal.html"
```

Для benign_minimal ожидается спокойный результат: score 0, verdict no_strong_static_anomalies, evidence_level none. Это хороший первый демонстрационный запуск.

4.2. Проверить подозрительный пример

```
set TARGET=%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\suspicious_tamper.json" ^
  --html "my_reports\suspicious_tamper.html" ^
  --csv "my_reports\suspicious_tamper_functions.csv"
start "" "my_reports\suspicious_tamper.html"
```

В HTML смотрите блоки Summary, Findings, Assessment и Functions. В JSON те же данные лежат в heuristic_analysis и assessment. CSV удобен для табличной сверки begin_rva, end_rva и статуса разбора unwind.

4.3. Сгенерировать Ghidra CSV для того же файла

```
"%TOOL%" "%TARGET%" --heuristics --ghidra-csv "my_reports\suspicious_tamper_ghidra.csv"
```

Этот CSV не заменяет обычный отчет. Он нужен именно для Ghidra-скрипта. В нем четыре ключевые колонки: begin_rva, end_rva, name, comment.

4.4. Запустить regression pack

```
python -m pip install -r requirements-dev.txt
python tests\run_regression_samples.py "%TOOL%"
```

Скрипт прогоняет встроенные образцы, проверяет JSON Schema и смысловые инварианты отчетов. Если все хорошо, по каждому sample будет [OK].

4.5. Где смотреть уже готовую evidence-базу проекта

reports\final_validation_2026-06-19\SUMMARY_RU.md	Итоговая сводка финальной проверки релиза 1.0.0.
reports\final_validation_2026-06-19\static_labeled	Отчеты по синтетическим статическим бинарникам с известными вмешательствами.
reports\final_validation_2026-06-19\static_real	Статическая проверка реального benign-корпуса.
reports\final_validation_2026-06-19\dynamic_smoke	Четыре контрольных динамических сценария VEH, VCH, UEF и runtime function table.
reports\final_validation_2026-06-19\dynamic_full	Полная динамическая evidence-база: JSON, CSV, REPORT.md и логи запусков.

> В финальной evidence-папке в основном лежат JSON, CSV, MD и LOG. HTML-отчеты создаются командой --html для выбранного PE-файла. Это нормально: HTML обычно генерируют под конкретный демонстрационный образец.

5. Динамический режим на Windows

Динамический режим включается опцией --dynamic. В этом режиме analyzer запускает target как дочерний процесс под отладчиком и собирает runtime evidence: исключения, VEH/VCH/UEF, обработчики, RtlAddFunctionTable, RtlInstallFunctionTableCallback, trace после handler hit.

> Не запускайте неизвестные файлы с --dynamic на основной системе. Для чужих EXE используйте изолированную Windows VM без важных данных. DLL/PYD напрямую динамически запускать нельзя: им нужен отдельный host process.

5.1. Обычный динамический запуск с HTML

```
set TARGET=C:\путь\к\target.exe
"%TOOL%" "%TARGET%" --heuristics --dynamic ^
  --timeout-ms 15000 ^
  --trace-steps 64 ^
  --json "my_reports\dynamic_report.json" ^
  --html "my_reports\dynamic_report.html"
start "" "my_reports\dynamic_report.html"
```

timeout-ms - жесткий лимит времени в миллисекундах. trace-steps - сколько single-step адресов собрать после попадания в handler breakpoint. Для обычной демонстрации 15000 и 64 достаточно.

5.2. Передать аргументы целевой программе

```
"%TOOL%" "%TARGET%" --heuristics --dynamic --json "my_reports\with_args.json" ^
  --html "my_reports\with_args.html" -- --input sample.dat --mode test
```

Все, что идет после отдельного разделителя --, передается уже target-программе, а не analyzer. Проект редактирует эти аргументы в report metadata, чтобы случайно не сохранить секреты вроде паролей.

5.3. Собрать и запустить dynamic smoke tests

Эти тесты нужны для демонстрации, что динамический collector видит VEH, UEF, VCH и runtime function table APIs. Сначала собираются специальные тестовые EXE в tests, затем запускается smoke pack.

```
cd /d "%PROJECT%"
cd tests
build_tests_msvc.bat
cd ..
tests\run_dynamic_smoke_windows.bat "%TOOL%"
```

Скрипт пишет tests\veh_dynamic_report.json, tests\uef_dynamic_report.json, tests\vch_dynamic_report.json, tests\rtl_dynamic_report.json и соответствующие *_dynamic_console.txt. В конце должно быть Dynamic smoke validation: OK.

5.4. Как читать dynamic-блок в JSON/HTML

- enabled=true означает, что пользователь включил --dynamic.

- ran=true означает, что динамическая сессия действительно стартовала.

- ok=true означает, что collector завершился без внутренней ошибки.

- timed_out=false означает, что target не был остановлен по timeout.

- target_exited=true означает, что целевой процесс завершился наблюдаемым образом.

- events показывает структурированную timeline: исключения, регистрации handlers, runtime table APIs и другие события.

- assessment.static_dynamic_correlation показывает, есть ли связь между статическими findings и runtime evidence.

6. Ghidra: как добавить данные EH/Unwind Analyzer

Ghidra-интеграция состоит из двух частей. Сначала analyzer создает специальный CSV через --ghidra-csv. Потом скрипт ghidra\import_unwind_csv.py импортирует CSV в открытую программу Ghidra.

6.1. Что делает Ghidra-скрипт

Создает функции	По begin_rva/end_rva скрипт создает функции в текущей программе Ghidra, если их еще нет.
Обновляет функции	Если функция уже есть, скрипт может обновить body и добавить комментарий.
Сохраняет ручные имена	Импорт осторожный: user/imported names не затираются без необходимости.
Добавляет комментарии	Перед функцией появляется pre-comment с префиксом [EH/Unwind Analyzer].
Ставит bookmarks	Если в комментарии есть parse failed или anomalies=, ставится Warning bookmark категории EH/Unwind Analyzer.
Пропускает конфликты	Если диапазон пересекается с чужой функцией или лежит вне памяти, строка CSV пропускается и печатается [skip].

6.2. Сгенерировать CSV для Ghidra

CSV обязательно должен быть создан для того же EXE, который потом открыт в Ghidra. Нельзя импортировать CSV от одного файла в другой файл: адреса и диапазоны не совпадут.

```
cd /d "%PROJECT%"
mkdir my_reports
set TARGET=%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --ghidra-csv "my_reports\suspicious_tamper_ghidra.csv" ^
  --html "my_reports\suspicious_tamper.html" ^
  --json "my_reports\suspicious_tamper.json"
```

Проверьте, что CSV создан:

```
type "my_reports\suspicious_tamper_ghidra.csv"
```

В первой строке должен быть заголовок begin_rva,end_rva,name,comment. Если файла нет, сначала смотрите ошибку analyzer в консоли.

6.3. Создать проект Ghidra

- Запустите Ghidra.

- Выберите File -> New Project.

- Выберите Non-Shared Project.

- Задайте имя, например EH_Unwind_Demo.

- Папку проекта Ghidra можно создать рядом с BKP, но не внутри build.

6.4. Импортировать EXE в Ghidra

- В главном окне Ghidra выберите File -> Import File.

- Выберите тот же TARGET, для которого был создан CSV.

- Убедитесь, что Ghidra определила формат как Portable Executable (PE).

- Нажмите OK.

- Дважды кликните по импортированному файлу, чтобы открыть CodeBrowser.

- Когда появится вопрос Analyze?, нажмите Yes или Analyze Now.

- Оставьте стандартные анализаторы включенными и нажмите Analyze.

Если Ghidra сама уже нашла часть функций, это нормально. Импортер не обязан создавать все заново: он может обновить существующие функции и добавить комментарии.

6.5. Подключить папку со скриптом import_unwind_csv.py

Скрипт лежит в папке проекта: ghidra\import_unwind_csv.py. Эту папку нужно добавить в Script Manager один раз.

- В CodeBrowser откройте Window -> Script Manager.

- Нажмите Manage Script Directories. В разных версиях Ghidra это кнопка со списком папок или маленькая иконка рядом с каталогами скриптов.

- Добавьте папку %PROJECT%\ghidra.

- Закройте окно управления директориями.

- В списке скриптов найдите import_unwind_csv.py. Обычно он находится в категории Analysis.

- Если скрипт не появился, нажмите Refresh в Script Manager или закройте и снова откройте Script Manager.

6.6. Запустить импорт CSV

- В Script Manager выделите import_unwind_csv.py.

- Нажмите Run Script.

- В появившемся окне выбора файла укажите my_reports\suspicious_tamper_ghidra.csv.

- Дождитесь завершения. Внизу в Console появится строка результата.

Пример нормального результата в консоли Ghidra:

```
EH/Unwind import complete: created=... updated=... renamed=... commented=... bookmarks=...
    skipped=0
```

created	Сколько новых функций создал скрипт.
updated	Сколько уже существующих функций обновил.
renamed	Сколько функций переименовал в импортированное имя, если это было безопасно.
commented	Сколько функций получили комментарий [EH/Unwind Analyzer].
bookmarks	Сколько warning bookmarks поставлено на parse failures или anomaly comments.
skipped	Сколько строк CSV пропущено из-за конфликта адресов, диапазонов или памяти.

6.7. Что смотреть в Ghidra после импорта

- Window -> Functions: список функций должен содержать импортированные/обновленные диапазоны.

- Listing: перед функциями должны появиться pre-comments с префиксом [EH/Unwind Analyzer].

- В комментариях ищите UNWIND v1/v2, flags, prolog, stack, frame, handler_rva и сообщения parse failed.

- Window -> Bookmarks: warning bookmarks категории EH/Unwind Analyzer показывают проблемные unwind-записи.

- Сравнивайте адреса так: Ghidra address = ImageBase + begin_rva. Для PE64 часто ImageBase равен 0x140000000.

Пример сверки RVA и адреса Ghidra:

```
begin_rva = 0x000013F0
ImageBase = 0x140000000
Ghidra address = 0x140000000 + 0x000013F0 = 0x1400013F0
```

6.8. Типовые проблемы Ghidra

Скрипт не виден	Не добавлена папка ip_full\ghidra в Script Manager. Добавьте ее через Manage Script Directories и нажмите Refresh.
CSV не подходит	CSV создан для другого EXE. Сгенерируйте --ghidra-csv заново для именно того файла, который открыт в Ghidra.
skipped > 0	Чаще всего диапазон пересекается с уже существующей функцией или лежит вне загруженной памяти. Проверьте, что EXE и CSV совпадают.
created=0	Это не ошибка, если Ghidra уже нашла функции. Смотрите updated и commented.
Нет bookmarks	Bookmarks появляются только для parse failed/anomalies. Для нормального файла bookmarks может быть 0.
Адреса выглядят другими	CSV хранит RVA, а Ghidra показывает VA. Прибавьте ImageBase.

6.9. Мини-чеклист для защиты по Ghidra

- Сначала показать HTML-отчет: где score, findings и assessment.

- Потом показать CSV для Ghidra: begin_rva, end_rva, name, comment.

- В Ghidra открыть тот же EXE.

- Запустить import_unwind_csv.py.

- Показать строку Console: commented > 0 и skipped=0.

- Открыть Listing и показать комментарий [EH/Unwind Analyzer].

- Открыть Bookmarks, если выбран файл с parse failed/anomalies.

- Объяснить, что Ghidra CSV - это визуальная разметка static evidence, а --dynamic - отдельное runtime evidence.

7. Частые ошибки и быстрые исправления

'cl' is not recognized	Открыт обычный cmd. Откройте x64 Native Tools Command Prompt for VS 2022 или установите C++ workload.
cmake не найден	Установите CMake или добавьте CMake из Visual Studio в PATH.
ninja не найден	Для CMake presets нужен Ninja. Используйте Visual Studio generator или установите Ninja.
tool not found	Неверно задан TOOL. Проверьте путь через dir "%TOOL%" или выполните "%TOOL%" --version.
Failed to read file	Неверный TARGET, нет доступа или путь без кавычек при пробелах.
Unknown/invalid option	Опечатка в ключе. Сверьтесь с "%TOOL%" --help.
Invalid value for --timeout-ms	timeout-ms должен быть больше 0.
HTML не открывается	Проверьте, что файл создан. Используйте start "" "path\report.html".
JSON пустой или старый	Проверьте return code analyzer и stderr. Writer использует atomic replacement, поэтому при ошибке старый отчет может сохраниться.
Dynamic mode зависает	Уменьшите timeout-ms, используйте безопасный тестовый EXE, запускайте в VM.
Антивирус блокирует EXE	Для своих тестовых бинарников добавьте исключение или запускайте в изолированной VM. Не отключайте защиту для неизвестных файлов на основной системе.

7.1. Как быстро проверить, что все готово к демонстрации

```
cd /d "%PROJECT%"
"%TOOL%" --version
"%TOOL%" "%CD%\tests\samples\bin\benign_minimal.exe" --heuristics ^
  --json "my_reports\demo.json" ^
  --html "my_reports\demo.html" ^
  --csv "my_reports\demo.csv" ^
  --ghidra-csv "my_reports\demo_ghidra.csv"
start "" "my_reports\demo.html"
```

Если эта последовательность прошла, то базовая демонстрация готова: EXE запускается, HTML открывается, JSON/CSV созданы, Ghidra CSV готов для импорта.

8. Короткий сценарий показа проекта преподавателю или заказчику

1. Показать структуру папки

```
dir
```

2. Показать версию analyzer

```
"%TOOL%" --version
```

3. Собрать проект из исходников

```
build_msvc.bat
```

4. Запустить быстрый benign sample

```
"%TOOL%" "%CD%\tests\samples\bin\benign_minimal.exe" --heuristics
```

5. Создать HTML/JSON/CSV

```
"%TOOL%" "%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe" --heuristics --json
    "my_reports\s.json" --html "my_reports\s.html" --csv "my_reports\s.csv"
```

6. Открыть HTML

```
start "" "my_reports\s.html"
```

7. Создать Ghidra CSV

```
"%TOOL%" "%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe" --heuristics --ghidra-csv
    "my_reports\s_ghidra.csv"
```

8. В Ghidra импортировать EXE

```
File -> Import File -> выбрать тот же EXE -> Analyze Now
```

9. В Ghidra запустить скрипт

```
Window -> Script Manager -> import_unwind_csv.py -> Run Script -> выбрать s_ghidra.csv
```

10. Показать evidence-базу

```
reports\final_validation_2026-06-19\SUMMARY_RU.md
```

> Главная мысль для объяснения: проект не просто печатает score. Он строит проверяемый pipeline: PE parser -> RUNTIME_FUNCTION -> UNWIND_INFO -> heuristics -> JSON/HTML/CSV -> optional Windows dynamic evidence -> Ghidra visualization.